

sdpa-gmp-omp: threading a multiprecision SDP solver, and re-deriving its factorisation choice

sdpa-gmp-omp — technical record

2026-08-20

Abstract

`sdpa-gmp` solves semidefinite programs in arbitrary-precision arithmetic. It builds with OpenMP and barely benefits: across five SDPLIB problems, thirty-two cores buy upstream $1.71\times$. This document records what was changed and why. Three things are worth a reader’s attention. First, the dominant cost is not where intuition puts it: profiling assigns 66–80% of runtime to the Schur-complement build, and a proposal to parallelise per-block Cholesky factorisations — the author’s first instinct — does not appear in the profile at all. Second, threading a multiprecision solver is mostly a *reproducibility* problem, not a speed one; upstream’s threaded runs are not reproducible, and one problem varied between 50 and 80 interior-point iterations run to run. Third, the heuristic choosing between a dense and a sparse factorisation of the Schur complement was wrong, and wrong in a way that a retune would not have fixed: two of its four gates were driven by a single shared constant, and once that is split, one of them is not a heuristic at all but a provable consequence of another. The net effect is that one tunable replaces two, and 167 of 221 census instances stop paying a $2.6\text{--}9.8\times$ penalty.

Contents

1	The solver, and where the time goes	2
2	Kernel and threshold defects in the inherited code	2
3	What was threaded, and what that costs correctness	3
4	Measured effect of the threading work	3
5	The factorisation choice	4
5.1	Four gates	4
5.2	One constant doing two unrelated jobs	4
5.3	Gate 3 is not a heuristic	4
5.4	Why the old constant was wrong — and why not for the reason first published	5
5.5	Scope: this fork only	5
5.6	Not “sparse always wins”	5

6	Evidence	5
6.1	Census	5
6.2	Thread matrix	6
6.3	Precision coverage	6
6.4	Numerical acceptance	6
7	The two factorisation paths, and why the advice about them inverted	7
8	What the memory cap bounds, and the one refusal that is not about memory	7
9	Thread count is a property of the machine	7
10	Environment variables	8
11	Exit status	11
12	Sources	11

1 The solver, and where the time goes

At each interior-point iteration the solver forms and factorises the $m \times m$ Schur complement B (the “bMat”), then performs triangular solves against it. In arbitrary-precision arithmetic every scalar operation is a function call on a heap-allocated mantissa, so the constant factors that a floating-point solver hides become the whole story.

Profiling on `arch0` and `control5` attributes **66–80% of runtime to the bMat build**. This matters methodologically: the first optimisation proposed for this work was a per-block parallel Cholesky, which does not appear in the profile. It was abandoned on the evidence, and this document records it because the discarded hypothesis is part of the result.

2 Kernel and threshold defects in the inherited code

Three defects were found in upstream, each measured on both x86-64 and arm64.

1. **A dropped zero-skip guard.** `mplapack` 2.0.1 no longer carries `netlib dgemm`’s guard that skips work when a multiplier is zero. In multiprecision the skipped work is not cheap: up to a $2 \times$ *serial* regression.
2. **Unconditional OpenMP regions.** Parallel regions were entered regardless of problem size, costing up to $13 \times$ on small problems. A threshold mechanism exists upstream but is disabled behind `if (0)` and cannot simply be re-enabled: the `_ref` bodies it calls are absent, so it would not link.
3. **Non-reproducible reduction.** `Rdot_omp` combined partial sums under `omp critical`, making the summation order depend on thread scheduling. One problem varied 50–80 iterations run to run, and three returned a different objective between runs.

Defect 3 is the important one. In multiprecision the reassociation error is tiny, but the Newton path amplifies it: once one iterate differs, subsequent iterates diverge, and the iteration

count — and occasionally the reported objective — follows. A solver whose answer depends on thread scheduling cannot be used for work that must be reproducible.

3 What was threaded, and what that costs correctness

Four regions are threaded: the `bMat` build and assembly, the sparse (chordal) Cholesky, and the forward triangular solve.

The forward solve is **bit-identical** by construction: within a row the writes go to distinct destinations, so no reassociation occurs. The backward pass *is* a reassociation and is therefore **opt-in** behind `SDPA_SOLVE_BACKWARD`, off by default. Measured at the shipped chunk count on a real $m = 6067$ problem, the first backward solve agrees with the serial answer to 1.1×10^{-77} of the vector norm — about 1.3 ulp at 256 bits. Later solves diverge further for the reason given above, which is the Newton path and not the kernel.

One further mechanism deserves mention because it is a pure-speed knob with a surprising magnitude. `SDPA_SOLVE_MAX_TEAM` (default 32) caps only the forward solve’s team. That pass scatters across the whole solution vector, so once its team spans two sockets every per-row barrier waits on cross-socket writes; uncapped at 128 threads it measured $18.5\times$ slower than capped at 32. It cannot change results — the backward chunk count is fixed independently of the team — so the cap costs nothing.

4 Measured effect of the threading work

One AMD EPYC 7742 node, 32 threads, against upstream `ca110db` from a pristine clone built `--enable-openmp=yes` and swept over the same thread counts, so upstream is compared at its own best rather than against a serial build chosen for it. Same compiler, flags, bundled GMP source and `param.sdpa` (requested 200-bit, actual 256-bit). External wall clock, median of three repeats. Iteration counts are identical in all 70 paired runs, so every ratio is like-for-like and equals a per-iteration ratio.

	upstream	fork	ratio
5 SDPLIB problems, both at 32 threads	115.10 s	32.13 s	$3.58\times$
5 SDPLIB problems, fork vs upstream <i>serial</i>	196.27 s	32.13 s	$6.11\times$
one large problem (<code>12_min</code>), both at 32 threads	2046 s	246 s	$8.32\times$

The decomposition matters more than the totals. At one thread the fork is only $1.05\text{--}1.21\times$ faster, so **almost the entire advantage is scaling rather than kernel micro-optimisation**:

	1 $\rightarrow$ 32 threads	gain
upstream, five problems	196.27 $\rightarrow$ 115.10 s	$1.71\times$
fork, five problems	175.77 $\rightarrow$ 32.13 s	$6.1\times$
upstream, <code>12_min</code>	3951 $\rightarrow$ 2046 s	$1.93\times$
fork, <code>12_min</code>	3765 $\rightarrow$ 246 s	$15.3\times$

On `truss5` upstream threading buys nothing measurable at all: 12.29 s at one thread, 12.13 s at thirty-two. The fork is nominally *slower* on problems too small to time (`control1` at 0.23–0.30 s, and `8_min`); those are recorded rather than excluded.

Separately, on the sparse route the fork is $4.2\times$ faster at $5\times$ less peak memory than the dense route on an $m = 6067$ problem — a figure that belongs to §5 rather than here, because it is a property of the factorisation choice and not of the threading.

5 The factorisation choice

5.1 Four gates

The solver must choose, per problem, between factorising B densely and factorising it as a sparse chordal matrix. The inherited rule is a cascade of four gates, the first that fires deciding:

1. $m \leq 100$ or $\text{nBlock} \leq 5 \Rightarrow$ dense (too small to be worth it);
2. any block with $\text{nConstraint} > m\sqrt{\tau} \Rightarrow$ dense;
3. aggregate density $\rho_{\text{agg}} > \tau \Rightarrow$ dense;
4. ordered fill $\rho_{\text{fill}} > F \Rightarrow$ dense, else sparse.

Here $\rho_{\text{agg}} = \text{nnz}(\text{pattern}(B))/m^2$ is known before any ordering is computed, while $\rho_{\text{fill}} = (2\text{nnz}(L) - m)/m^2$ requires a symbolic factorisation and therefore costs one graph ordering. The shipped constants are $\tau = 0.25$ and $F = 0.40$.

5.2 One constant doing two unrelated jobs

Gates 2 and 3 are both derived from τ : gate 2 as $m\sqrt{\tau}$, gate 3 as τm^2 . They answer unrelated questions — gate 2 asks whether a single block is too dense to be worth treating sparsely, gate 3 asks whether the whole pattern is. Because they share a source constant, **neither can be retuned without moving the other**, and any attempt to recalibrate gate 3 silently changes gate 2’s block cutoff. The first step is therefore not a retune but a *split*: gate 2 is given its own constant, frozen at exactly the $0.5m$ that $m\sqrt{0.25}$ already evaluated to, so that gate 2’s behaviour is preserved bit-for-bit while gate 3 becomes free.

5.3 Gate 3 is not a heuristic

Lemma 1 (Containment). *Symbolic factorisation of a symmetric pattern only adds entries. In the symmetric-with-diagonal counting convention used by both gates, $\rho_{\text{fill}} \geq \rho_{\text{agg}}$ for every ordering.*

Corollary 1. *If $\rho_{\text{agg}} > F$ then $\rho_{\text{fill}} > F$, so gate 4 would return dense whatever ordering it computes. Deciding dense at that point cannot change the verdict; it only avoids paying for an ordering whose outcome is already determined.*

Corollary 1 replaces gate 3 entirely. It is no longer an independent policy with a constant of its own but the early, cheap form of gate 4, using gate 4’s own threshold F . The result is that **one tunable (F , unchanged at 0.40) replaces two**, and the rule’s only remaining decision is the one it can actually measure.

Lemma 1 is asserted at runtime as a hard error rather than trusted: if the two counts ever cease to be in the same units, the solver refuses. The assertion is on the *ordered path* only — an input taking the Corollary 1 exit returns before any fill count exists, so no runtime check applies to it, and the skip’s soundness there rests on the lemma and on the boundary fixtures.

5.4 Why the old constant was wrong — and why not for the reason first published

The natural explanation, and the one this project published for several days, is that $\tau = 0.25$ was calibrated when the sparse Cholesky was serial and became wrong once it was threaded. That explanation is **false**, and the measurement that falsified it was performed only because a reviewer insisted the predeclared thread matrix be completed rather than narrowed.

At *one thread* — the serial regime the constant was calibrated for — sparse still wins all four measured structures on both architectures:

structure	ρ_{agg}	Xeon $T=1$	EPYC $T=1$
$m = 2439$	0.262	$5.32\times$	$\geq 2.99\times$
$m = 4489$	0.367	$2.60\times$	$2.62\times$
$m = 5278$	0.293	$\geq 3.00\times$	$\geq 3.06\times$
$m = 6067$	0.259	$\geq 3.00\times$	$\geq 2.85\times$

Bounded entries are ones where the dense arm hit a wall cap, so the true margin is larger. Every dense arm was witnessed at 98–99% of one core. The two exact figures for $m = 4489$ — the structure at the top of the census density range, and so the least favourable case — agree across unrelated hardware to 0.8%.

So the threshold mis-routed these structures before threading existed. Threading *amplifies* the penalty to roughly $8\times$ by 32–64 threads, largely because the dense arm’s parallel efficiency collapses (99% at $T = 1$ to 15% at $T = 64$ on $m = 2439$) while the sparse arm keeps scaling. **Why** the constant is wrong in the serial regime is not established by this work, and no mechanism is asserted here.

5.5 Scope: this fork only

The chooser code in §5 exists identically in the `sdpa-dd` and `sdpa-qd` siblings, and the change was **not** ported to them — they still gate on the released constant. Nothing measured here is evidence about them, and not only because the code was untouched: `dd` works in double-double and `qd` in quad-double, while every measurement above is at 256 or 512 bits, and the relative cost of the two factorisations depends on precision.

5.6 Not “sparse always wins”

SDPLIB `truss5` has $\rho_{\text{fill}} = 1.0$ — a wholly dense factor — and dense is $1.4\times$ faster on it. The released rule sent it to dense and was right to. It is the boundary control against `truss6` at $\rho_{\text{fill}} = 0.329$, and the two bracket $F = 0.40$ from either side. A rule that tested anything other than fill would get one of them wrong.

6 Evidence

6.1 Census

Two corpora were enumerated with the chooser instrumented, recording for every problem which gate decided it and both densities. The bootstrap corpus is 221 input files over 10 distinct structures; 167 instances, across seven affected structures, were routed dense by the old rule.

The measured ρ_{fill} over that corpus spans 0.244–0.367. SDPLIB’s 92 problems take only two distinct ρ_{fill} values, 0.329 and 1.000, so the interval (0.367, 1.0) contains no member of either corpus — no measured problem sits where the rule’s verdict is in doubt. Notably, **no SDPLIB problem changes route** under the new policy, which is why the published benchmark tables are unaffected by it.

6.2 Thread matrix

The promotion condition predeclared a thread matrix, completed in full: EPYC 2×64 at $T \in \{1, 8, 32, 64, 128\}$ and Xeon 2×20 at $T \in \{1, 8, 20, 40\}$ — nine points, both architectures, 40 problem-labels, **every one sparse**, with no reversal anywhere. Labels come from a fail-closed analyser enforcing a pinned problem list, a minimum repetition count, arm status, memory vetoes, and a measured parallelism witness read from each dense arm’s resource usage. Two limits are stated rather than rounded away. $T = 128$ **carries a single problem** ($m = 6067$) where every other point carries four, so that cell is a direction and not a population. And the whole matrix is release-binary evidence with modes forced, which is a valid route comparison but not evidence about any particular build.

6.3 Precision coverage

All of the above is at 256-bit working precision except one problem, $m = 6067$, re-run at 512-bit, where sparse won by $\geq 8.00\times$. **One problem fixes the sign, not the shape**: it says the route preference does not invert at higher precision, and says nothing about whether the margin grows, shrinks or holds as precision rises. Relatedly, the OpenMP work thresholds — a separate mechanism from the chooser — were calibrated at roughly 256-bit cost, so the point at which entering a parallel region becomes profitable moves with precision and would need re-sweeping elsewhere.

6.4 Numerical acceptance

Selecting a different factorisation changes results in the last digits, so the change was gated on a solution comparison at full convergence on the four structures where the dense arm converges inside a practical wall:

structure	phase	iterations	$\max \Delta x / \ x\ _\infty$
$m = 2439$	pdOPT	23	4.183×10^{-44}
$m = 4489$	pdINF	31	2.583×10^{-44}
$m = 5278$	pdFEAS	37	6.298×10^{-38}
$m = 6067$	pdOPT	35	2.844×10^{-30}

Printed objectives were bit-identical and iteration counts equal in every case, across three different convergence outcomes, against a declared tolerance of 10^{-8} . On the remaining three affected structures the dense arm does not converge inside a practical wall, so no comparison is computable there; that impracticality is itself the retained finding.

7 The two factorisation paths, and why the advice about them inverted

B is factored by one of two routines, chosen at start-up from the problem’s sparsity:

path	routine	threaded
dense	Rpotrf (mplapack)	yes
sparse	Lal::getCholesky	yes, since the sparse-Cholesky work described in §3

The upstream rule minimised *operation count*. That was the right objective for a serial solver and stopped being right once both paths threaded, which is what §5 replaces.

The practical advice inverted, and older write-ups of this fork say the opposite. When only the dense path was threaded, forcing dense on one problem ($m = 7401$, 256-bit, 32 threads) was $2.42\times$ faster for $5.4\times$ the memory — so the guidance then was to force dense near the boundary. With the sparse path threaded, the same problem on the same node reverses: forcing sparse is faster *and* smaller, and the boundary problems are exactly the ones §5 re-routes. A reader finding the older advice should treat it as a statement about a solver that no longer exists.

8 What the memory cap bounds, and the one refusal that is not about memory

A dense B holds m^2 separately allocated GMP values — roughly 80 bytes each at 256-bit precision, 112 at 512 — so forcing dense on a large problem is expensive by construction. SDPA_BMAT_MAX_GB exists to bound that, and it is worth being precise about what it does and does not promise.

It bounds $m^2 \times$ bytes per element: **an estimate of one matrix**. It is not an rlimit, not a bound on peak resident set size, and not a guarantee about total process memory. The rest of the solver allocates too, and per-object GMP bookkeeping can exceed the fixed per-element allowance — in a measured $m = 7401$ run the estimate was 4.08 GiB against a 4.54 GiB process peak. It decides which algorithm is chosen; it does not police the allocator.

Two consequences follow, both deliberate. The cap binds on *every* route to a dense B , not only on an explicit request for one, so setting it alone is a valid way to stop the automatic policy choosing an allocation the machine cannot afford. And when an explicit dense request conflicts with a cap the request exceeds, the cap wins — a limit that yields to a preference is not a limit.

Separately, and not a memory condition at all: $m \geq 46341$ **is refused outright**, because m^2 then exceeds INT_MAX and the element count of the allocation cannot be represented. No amount of memory changes this, and it is reported as a refusal rather than an allocation failure so that it cannot be mistaken for one.

9 Thread count is a property of the machine

“Match OMP_NUM_THREADS to physical cores” is ordinary advice and is wrong here on a large two-socket node, because the Cholesky’s per-update barrier ends up waiting on cross-socket memory traffic. The two architectures measured disagree, which is the point:

node	crossing the socket boundary	effect
2×64 -core EPYC 7742	$64 \rightarrow 128$ threads	$2.65\times$ slower
2×20 -core Xeon	$20 \rightarrow 40$ threads	$1.04\text{--}1.30\times$ faster

The crossing pays on a small two-socket box and costs badly on a big one. Separately, across the whole Xeon matrix — 24 sparse-route runs — every retained objective and iteration count is bit-identical, so tuning the thread count for speed costs nothing in reproducibility.

10 Environment variables

All are unset by default. With them unset, every `SDPA_SOLVE_*` variable leaves results unchanged; `SDPA_BMAT_MODE` is the exception discussed in §5, since leaving it unset now selects the derived chooser. In normal use nothing here needs setting — the one knob worth choosing deliberately is `OMP_NUM_THREADS`, and `SDPA_BMAT_MODE=legacy` is what reproduces a pre-2026-08 result bit-for-bit.

An asymmetry inside the group, deliberate and stated because it otherwise reads as a bug: an *empty* `SDPA_BMAT_MODE` is treated as unset, whereas an *empty* `SDPA_BMAT_MAX_GB` is refused. An empty cap almost always means an unset shell variable was expanded into it, and silently running with no ceiling when the user asked for one is the dangerous reading; an empty mode has no such failure direction.

variable	values	effect
<code>SDPA_BMAT_MODE</code>	<code>auto</code> (default), <code>dense</code> , <code>sparse</code> , <code>fill</code> , <code>legacy</code>	Which factorisation of the Schur complement (<code>bMat</code>) is used. <code>auto</code> changed meaning on 2026-08-18 — it is now the derived chooser, and <code>legacy</code> restores the pre-2026-08 one exactly , which is what to set to reproduce an older result bit-for-bit. <code>fill</code> is a synonym for <code>auto</code> . <code>dense/sparse</code> force a route past the gates. Forcing a route changes results in the last digits, by construction. Rationale, the gate derivation and the measurements: doc/technical.pdf , "The factorisation choice" and "Evidence".
<code>SDPA_BMAT_LOG</code>	any non-empty value other than 0	Prints each selection gate with its numbers, and the verdict, to stdout and to the <code>-o</code> file.
<code>SDPA_BMAT_MAX_GB</code>	GiB, e.g. 64	Opt-in ceiling on the dense <code>bMat</code> , however it was selected. Unset means no ceiling.
<code>SDPA_SPCHOL_MODE</code>	<code>auto</code> (default), <code>serial</code> , <code>parallel</code> , <code>legacy</code>	Overrides how the <code>sparse</code> Cholesky runs. <code>legacy</code> is the pre-optimisation expression, kept as a validation oracle.
<code>SDPA_SPCHOL_LOG</code>	any non-empty value other than 0	Prints the sparse-Cholesky path, team size, workers that performed updates, update counts, and the effective admission thresholds.
<code>SDPA_SPCHOL_DIGEST</code>	any non-empty value other than 0	Prints a 64-bit fingerprint of the factor, with its record and byte counts, once per factorisation. Strong evidence of a change, not a proof of identity — for that use the dump below. Costs $O(\text{nnz})$ string conversions, so it is off by default.

variable	values	effect
SDPA_SPCOL_DIGEST_DUMP	a file path	Appends the canonical length-framed factor stream — type, dimensions, sparse counts, row extents, and every element’s row, column, precision, sign, exponent and exact base-16 mantissa — so two runs can be compared exactly with <code>cmp</code> , rather than by hash equality. It appends , so remove the file before each run or two runs will concatenate into one and compare unequal for the wrong reason. One record per factorisation and one entry per non-zero: a 33 MB stream for a 172×172 fixture, and gigabytes for a real problem. A failed write is a hard error rather than a silently truncated file.
SDPA_SOLVE_MODE	<code>auto</code> (default), <code>serial</code> , <code>parallel</code>	Overrides how the forward triangular substitution runs. These settings describe the sparse triangular solve; on a problem whose <code>bMat</code> is dense there is no such pass, and a <i>forced</i> <code>parallel</code> is refused rather than silently ignored. Threading it is bit-identical — within a row the columns are distinct, so the row’s writes go to distinct destinations while rows still run in order — so <code>auto</code> enables it and results do not change.
SDPA_SOLVE_BACKWARD	<code>serial</code> (default), <code>auto</code> , <code>parallel</code>	Threads the backward substitution. Opt-in, because it reorders a finite-precision sum and so changes the answer in the last digits. What you get instead is reproducibility: the row is split into a fixed number of chunks independent of the thread count, so 1, 8, 32 and 128 threads all produce identical bits. Iteration counts may differ from a serial run. Measured agreement with the serial answer, and why later solves diverge further: <code>doc/technical.pdf</code> , “What was threaded, and what that costs correctness”.
SDPA_SOLVE_MAX_TEAM	threads, default 32; 0 disables the cap	Caps the team used by the triangular solves only. This is a guard against a large regression, not a tuning knob : the forward pass scatters across the whole solution vector, so once the team spans two CPU sockets every per-row barrier waits on cross-socket writes. Measured on a 2×64 -core EPYC 7742, the uncapped forward pass at 128 threads ran $18.5 \times$ slower than the same pass capped to 32, turning the whole solve into $2.2 \times$ <i>slower than serial</i> . The pass saturates by 16 threads, so the cap costs nothing. The default 32 is one machine’s socket width and may be wrong for yours — raise it if your sockets are wider. It changes speed only: the backward chunk count is fixed independently of the team, so capping can never change results.
SDPA_SOLVE_LOG	any non-empty value other than 0	Prints, once per run, which passes were threaded, the team size and cap, the chunk count, the problem’s row count, non-zeros and mean row length, and the reason if a pass stayed serial.
SDPA_SOLVE_MIN_ROW	mean off-diagonals per row, default 256	Admission gate: below this the solve stays serial, because a per-row barrier costs more than a short row’s work. Break-even at 32 threads is a mean row near 56, so the default asks a problem to be clearly worth threading.
SDPA_SOLVE_MIN_ROWS	rows, default 64	Admission gate on the number of rows.

variable	values	effect
SDPA_SOLVE_CHUNKS	chunks per row, default 64	Fixed chunk count for the threaded backward pass. Changing it changes the answer (differently, not more wrongly); it is fixed and thread-count-independent so that results stay reproducible.
SDPA_SOLVE_MIN_CHUNK	off-diagonals, default 4 x SDPA_SOLVE_CHUNKS	Rows shorter than this are computed by one thread in exactly the serial order, so short rows are unaffected by the reordering.
SDPA_SOLVE_DIGEST	any non-empty value other than 0	Prints a 64-bit fingerprint of the solved vector once per solve call.
SDPA_SOLVE_DUMP	a file path	Appends the canonical stream of the solved vector — every element’s precision, sign, exponent and exact base-16 mantissa — so two runs can be compared exactly with cmp . The printed solution carries too few digits to show a last-bit change, which is the only change a reordering makes, so this is the oracle for the solve. It appends : remove the file before each run. Tens of MB for a real problem.
SDPA_SOLVE_PROFILE	any non-empty value other than 0	At exit, prints the solve broken into index-build, copy, forward and backward, with mean, min, max and standard deviation across calls, and how many calls actually ran each pass in parallel. Writes to SDPA_SOLVE_PROFILE_OUT if set, otherwise stderr.
SDPA_OMP_MIN_SPCOL_TOTAL	matched updates, default 200000	Whole-factor admission threshold: below it the factorisation stays serial.
SDPA_OMP_MIN_SPCOL_WORK	matched updates, default 20000	Per-pivot threshold; a pivot below it is done by one thread.
SDPA_OMP_MIN_SPCOL_WIDTH	target rows, default 8	Per-pivot width threshold, applied together with the one above.
SDPA_BMAT_ASM_MODE	auto (default), serial, parallel	How the sparse Schur-complement assembly (Make bMat) runs, once the sparse path is selected. parallel forces past the per-block gate; a build without OpenMP refuses it.
SDPA_BMAT_ASM_MIN_PAIRS	pairs, default 4000	Per-block admission gate for the threaded assembly; blocks below it stay serial under auto .
SDPA_BMAT_ASM_SCRATCH_MB	MiB, default 4096	Budget for the extra workers’ private scratch (two block-sized matrices each); bounds the team per block. 0 = unbounded.
SDPA_BMAT_ASM_LOG	any non-empty value except 0	Per block: the admission decision with its reason, and after execution the requested/actual team, distinct workers that did pair work, and group/pair totals.
SDPA_BMAT_ASM_PROFILE	any non-empty value except 0	Per solve: seconds in the explicit per-group precompute and in the lazy $G = X \cdot F$ gemm inside calF2 , which no other timer isolates.
SDPA_BMAT_ASM_CENSUS	1, or exit to stop after printing	Structure of the assembly: per block, pairs, i-groups, formula mix, dense-Aj incidence. exit makes corpus sweeps cheap.

variable	values	effect
SDPA_BMAT_ASM_CENSUS_VERBOSE	any non-empty value except 0	With the census: one line per group and per pair, naming the formula and each operand's representation. For locating fixtures, not for production logs.
SDPA_BMAT_ASM_DUMP	a file path	Appends the canonical stream of the assembled bMat, before factorisation — the assembly's own exact oracle, since two different matrices can round to the same factor. Same append/size cautions as the factor dump.
SDPA_BMAT_ASM_DIGEST	any non-empty value except 0	Prints the assembled matrix's fingerprint with record/byte counts, once per assembly.

11 Exit status

Scripts that loop over problems can rely on the exit code.

outcome	exit
solver ran to any stopping condition (pdOPT, pdFEAS, pFEAS, dFEAS, pdINF, pINF, dFEAS, pFEAS, dINF, pUNBD, dUNBD, noINFO)	0
iteration limit reached	0
infeasibility / unboundedness detected	0
malformed input, unreadable file, invalid parameter	1 with a diagnostic (line-numbered for data files)
numerical failure with nothing valid to print – no iteration completed, or the updated X/Z left the cone and the rollback could not be refactored	2 , solveStatus = FAILURE in the result file, no solution section
recoverable late failure after k good iterations – a Schur factorisation failure, or an X/Z update that left the cone and was rolled back ; the last valid iterate is printed and labelled	3 , solveStatus = PARTIAL , failureIteration = k in the result file

Infeasibility and the iteration limit are valid mathematical results, not errors. Upstream exited 0 on *every* path – including fatal errors – so a crashed run was indistinguishable

12 Sources

Figures in this document come from two places. Those in §4 are reproducible from this repository: `BENCHMARKS.md` carries the full tables and the methodology, and `bench/` the harness that produced them.

Everything in §6 — the censuses, the thread matrix, the label sets, the fail-closed analysers and the numerical-acceptance outputs — lives in a separate working repository that is not published alongside this fork, because it consists largely of raw solver output and cluster-specific deployment scripts. What is reproducible here without it: the chooser's own reasoning is visible at runtime with `SDPA_BMAT_LOG=1`, which prints each gate with its numbers and the verdict; and `tests/bmat_gate_fixtures.sh` in this repository asserts the whole decision cascade against

hand-computable fixtures on both sides of every gate, under both the current and the pre-2026-08 policies, so the rule derived in §5 is checkable from a clone.

An earlier LaTeX note derived Lemma 1 alone. Its abstract still carries the causal claim retracted in §5.4, and this document supersedes it.